

Cómo usar el Arquitecto (y el proyecto que te deja montado)

Guía de uso completa. El instalador está en `INSTALAR.md`. Si este documento y los archivos del sistema alguna vez difieren, **ganan los archivos** (`~/ .claude/skills/arquitecto/`). Versión 2026-07-06.

1. Qué es, en una frase

El Arquitecto es el interlocutor con el que **charlás una app antes de construirla**: te entrevista de a una pregunta, piensa lo que vos no ves venir (roles, listas configurables, multi-tenant 🚩, i18n 🚩), escribe el plano (SPEC-0) con una puerta de aprobación real, y te deja el proyecto **montado y en régimen** — `CLAUDE.md`, documentación, skills `/inicio` y `/cierre`, hooks y primer commit.

Qué NO es: no construye la app (eso lo hace otra sesión, con contexto limpio), no es para features chicas ni para el trabajo del día a día.

2. Los 3 momentos — cuándo sí, cuándo no

Momento	¿Arquitecto?	Qué hacés
Proyecto nuevo de cero	✅ Sí — es SU caso	<code>/arquitecto</code> en cualquier chat (caso A, abajo)
Feature grande o riesgosa en una app que ya anda	⚠️ Todavía no (Modo B llega en v2.1) — pero el patrón manual funciona HOY	Caso B, abajo: charlar → spec en formato SPEC-0 → sesión fresca lo ejecuta
Trabajo del día a día (fix, ajuste, feature chica)	❌ NO	El ciclo de siempre: <code>inicio</code> → pedís → verificás → <code>cierre</code>

Regla de bolsillo: si la feature se hace en una tarde y no toca permisos/datos/plata, **no hace falta spec** — pedila directo. Si "hacerla de a poquito puede salir mal", va con spec.

3. Caso A — Proyecto nuevo de cero (la película completa)

Ejemplo: se te ocurre "una app para llevar los gastos de la casa".

1. **Abrís un chat nuevo** (en cualquier carpeta) y decís: `/arquitecto quiero una app para llevar los gastos de casa`
2. **Crea la carpeta y el borrador.** `<carpeta de proyectos>/GastosCasa/SPEC-0.md` en estado BORRADOR. Desde ahí, cada respuesta tuya queda grabada en disco: si te vas a mitad de charla, no se pierde nada.
3. **Te entrevista** — una pregunta por vez, opciones con "(Recomendado)", en criollo: ¿web o celular? ¿quién la usa? ¿login? Y las dos ⚠ que SIEMPRE pregunta porque cambiar de opinión después es carísimo: ¿multi-idioma algún día? ¿clientes externos separados (multi-tenant)? Mientras charlan, manda ayudantes a investigar en paralelo (librerías vigentes del stack, apps parecidas).
 - **Atajo:** decí "dale con los defaults" y salta directo a confirmar solo las dos ⚠ y el tipo de app. Todo lo demás lo asume razonable y lo anota en **Supuestos** (lo corregís de un vistazo en el spec).
4. **Concerns:** te muestra la checklist con todo prendido por default — roles/permisos, listas configurables desde panel, manejo de errores, logging, auditoría (obligatoria si toca plata). Vos apagás lo que no aplique. *Tu dolor #1 — "me doy cuenta tarde de lo que la app necesitaba" — resuelto el día cero.*
5. **El plano y la puerta:** entra en modo solo-lectura (el sistema le IMPIDE escribir), te presenta 2-3 enfoques con pros/contras y su recomendación, y arma el SPEC-0: qué entra, **qué NO entra**, criterios verificables, supuestos, riesgos ⚠. Si el proyecto toca plata, permisos o datos de terceros, primero lo ataca el agente **red-team** y te trae lo que sobrevivió. Aprobás como un presupuesto — tu OK es una puerta real del sistema.
6. **Montaje** (solo con tu OK): SPEC-0 pasa a READY → scaffolding oficial del stack → git init + .gitignore correcto → CLAUDE.md lleno con lo que charlaron → docs (handoff, TODO con las primeras tareas, changelog) → skills `/inicio` y `/cierre` → hooks → primer commit. Te muestra la checklist con evidencia (✅ por ítem).
7. **El handoff** — te da el prompt exacto, listo para copiar:

Abrí un chat nuevo en `<carpeta del proyecto>` y pegá: `inicio – ejecutá el SPEC-0 (SPEC-0.md, está READY)`

8. **Y se apaga.** El chat nuevo construye la app siguiendo el plano; vos verificás y cerrás con `cierre`. De ahí en más, el ciclo de siempre.

Duración típica: 15-30 min de charla + el montaje.

4. Caso B — Feature grande en una app que YA anda ★

La pregunta clave: "estoy en un chat del proyecto, ya investigué, ya tengo la idea... ¿sigo acá o abro un chat nuevo?"

Respuesta: te quedás en ESE chat para pensar y escribir el spec. Abrís uno nuevo SOLO para construir. La investigación que ya hiciste es oro — no la tires cambiando de chat.

El flujo, paso a paso:

1. **En el chat donde venís charlando** (con la investigación ya hecha), decile:

Cristalizá todo esto en un spec en formato SPEC-0 — guardalo en `docs/SPEC-<nombre-de-la-feature>.md`. Es una feature sobre la app existente: dejá explícito qué cambia y **qué NO se toca**.

2. **Revisás el spec en español**: alcance, fuera de alcance, supuestos, riesgos. Pedís cambios hasta que esté como querés. Si la feature toca plata/permisos/datos:

Antes de marcarlo READY, tirale el agente redteam-spec y traeme lo que sobreviva.

3. **Se marca READY** → cerrás la sesión como siempre (`cierre` — el spec queda commiteado).
4. **Chat NUEVO en el proyecto** → `inicio` — ejecutá el spec `docs/SPEC-<nombre>.md` (está READY) La sesión fresca construye con contexto limpio, verificás con `/smoke`, `cierre`.
5. Cuando la feature está implementada, `/cierre` archiva el spec a `docs/archive/` solo.

¿Por qué no construir en el mismo chat donde pensaste? Dos razones medidas: (a) el chat de diseño está lleno de ideas descartadas y callejones — la sesión fresca lee SOLO el plano aprobado y no se confunde; (b) es la puerta de control: entre pensar y construir estás VOS aprobando un archivo. Es el mismo principio del Arquitecto, aplicado a mano.

Si todavía no investigaste nada: la skill `brainstorming` (instalada global) te hace la entrevista de la feature — una pregunta por vez, igual que el Arquitecto. La regla ya está en tus CLAUDE.md: venga de donde venga la charla, **el spec sale SIEMPRE en formato SPEC-0**.

En v2.1 esto será `/arquitecto` a secas dentro del proyecto (Modo B): explorará tu código primero y charlará anclado a lo que existe. Hasta entonces, el patrón manual de arriba es exactamente lo que hizo Perseo con sus features grandes — funciona.

5. El proyecto que te deja montado — cómo se usa después

El Arquitecto te entrega un proyecto **en régimen**: el mismo sistema que corre en Perseo y Hermes. Tu día a día ahí:

- **Abrís chat en la carpeta del proyecto** → decís `inicio` → Claude ya sabe dónde están parados (el hook le inyectó el estado) → te confirma en 3 líneas → le das la misión.
- **Trabajás**. Las reglas del CLAUDE.md operan solas: evidencia real o "NO VERIFICADO", tus datos son intocables sin tabla "cambio/preservo", tareas largas en background.
- **Terminás** → `cierre` → docs al día, commit, push, y el chat se descarta. **Cambió la misión** → **cambiá el chat**.
- `/smoke` y `/deploy` **no existen todavía** en un proyecto recién nacido — a propósito (regla de 3+: una skill nace de un ritual repetido, no "por las dudas"). El día que repetiste 3 veces el mismo ritual de probar

o deployar, pedile a Claude que la cree — los contratos ya están esperando en los templates del Arquitecto.

5b. El Equipador (`/arquitecto-skills`) — la skill hermana

El Arquitecto monta **proyectos**; el Equipador equipa **máquinas**. Gestiona las skills globales según un **menú curado** (con orígenes y razones, incluso de lo descartado):

- **"preparame esta máquina"** → censa qué falta del menú, te ofrece el Tier 1 con multiSelect, clona fresco de los repos oficiales e instala. Los plugins que necesitan comandos tuyos (`/plugin` , tokens) te los deja listos para pegar.
- **"actualizá las skills"** → re-clona y compara; si vos editaste algo local, te muestra el diff y pregunta — nunca pisa a ciegas.
- **"qué skills tengo"** → auditoría + propuesta de poda (lo que no se usa, afuera).
- **"agregá/investigá tal skill"** → investiga (repo vivo, licencia, colisiones), y si pasa la curaduría entra AL MENÚ primero, con razón escrita. El menú evoluciona; nada entra "por las dudas".

6. Qué hace bien y qué no (los límites, honestos)

Hace bien: pensar los concerns que olvidarías · elegir stack con evidencia del día (lo que puede envejecer está marcado `[VERIFICAR]` y lo re-chequea al usarlo) · el gate de aprobación · el montaje completo · retomar borradores abandonados · el red-team de specs sensibles.

No hace (todavía o nunca):

- ❌ Construir la app — NUNCA lo va a hacer; es el diseño.
 - ❌ Features sobre apps existentes (Modo B, v2.1) — usá el Caso B manual.
 - ❌ Consultorio de prompts/skills (Modo C, v2.1).
 - ⚠️ Si le pedís que "ya que está, programe algo": se va a negar y explicarte por qué.
-

7. Trucos y preguntas frecuentes

- **"¿Mismo chat o chat nuevo?"** — Para PENSAR (proyecto o feature): el chat donde está el contexto de la charla. Para CONSTRUIR: SIEMPRE chat nuevo con el spec como única fuente.
- **"Me fui a mitad de entrevista"** — Nada se perdió: `/arquitecto` de nuevo y te ofrece retomar donde quedaron (el borrador vive en disco).
- **"Quiero ir rápido"** — *"dale con los defaults"*: solo confirma las 2 ⚠️ y asume el resto en Supuestos.
- **"¿Y si me arrepiento del stack a mitad de entrevista?"** — Decilo nomás: el borrador se actualiza; nada está construido hasta que aprobás el plano.

- **"¿Dónde quedó el spec?"** — Proyecto nuevo: `SPEC-0.md` en la raíz del proyecto. Feature: `docs/SPEC-<nombre>.md`. Implementado → `docs/archive/` (lo archiva `/cierre`).
 - **"El Arquitecto se equivocó en algo"** — Se edita como cualquier skill: chat nuevo, "*con writing-skills, corregí tal cosa en ~/.claude/skills/arquitecto*". Y al cerrar una mejora grande: snapshot a `legacy/snapshots/` (regla del LEEME).
 - **"¿Sirve en la otra PC?"** — Sí: `arquitecto-portable.zip` → `INSTALAR.md` → pegar el prompt. Una sola línea se adapta por máquina (la carpeta de proyectos).
-

Estado al 2026-07-06: Modo A construido y validado adversarialmente (6 tests, 8 fixes). Pendiente: la prueba de fuego real. Modos B y C: v2.1. El diseño completo vive en `PROPUESTA-VIBE-KIT-V2.md` (Guía de vibe coding); el método madre en `PLAYBOOK-MAESTRO.md`.